What problem is your capstone agent designed to solve, and why does it require a multi-agent approach rather than a single agent? For instance, a research-heavy task may benefit from a planner agent, a researcher agent, and a writer agent working together.

- My agent is designed to solve the time consuming and broad scope of planning travel itineraries. There are many branches and factors that come into play when tackling this task. One of the trickiest aspects is to ensure each item aligns with each other. For instance, when booking flights, hotels, and rental cars, we have to ensure that the car is available to rent after the flight lands. Too early and you'll pay extra. Too late and you'll be stuck waiting at the airport. The same problem is present during hotel reservations. Furthermore, if there are certain attractions or activities you want to do, the agent has to be able to discern which ones you're able to attend (ensure no overlapping times), budget constraints, preferences, group sizes, etc.
- This makes the problem suitable for a multi-agent approach because the workflow contains several distinct but interdependent tasks. Specialized agents can handle transportation, activities, and dining while coordinating shared constraints such as timing, location, budget, and group preferences.

How many agents will your system include, and how did you determine the appropriate number? Consider task complexity, role separation, and diminishing returns from adding more agents.

- There will be three agents. Each agent takes care of a core component of a typical travel plan. Similar tasks have been grouped together to relieve some complexity. While some different tasks can be split into their own agent, it adds unnecessary complexity.

What are the defined roles and responsibilities of each agent?

- There would be a travel arrangement agent in charge of flights, hotels, and rental cars. Its main role is to find the basic essentials of every travel plan and ensure they all align. They need to be in the correct location, similar time, correct sizing (big car for a big group, enough reservations for the entire crew, etc.), and checks all the preferences of the group (e.g., no stop flights).
- The next agent would be for attractions and activities.This agent would be focused on finding the best tourist spots, must see activities, hidden gems, and similar attractions.
- The last agent looks for restaurants. While this could be intertwined with the previous agents, given the vast amount of options, it deserves its own agent. It must look for appropriate restaurants given the group. It has to be able to meet the needs of each individual (dietary restrictions, size of group, group age, etc.).

What coordination strategy will govern how agents interact (for example, sequential, iterative, graph-based, or hybrid)? Sequential workflows resemble assembly lines, while graph-based workflows resemble project networks with dependencies.

How will agents communicate and exchange information throughout the workflow? One-way communication for efficiency, two-way for validation, or brainstorming for exploring multiple paths.

- The strategy will follow a hybrid approach, especially depending on the initial user prompt. If the user already has a strong idea in mind for the travel plan, then a more sequential approach will be used. First the travel arrangement agent will take over and then the other two will build around that. The attractions and restaurant agents will follow a more graph based approach where they'll work in parallel and iteratively communicate with each other. They will follow a more brainstorming and cooperative approach. If, however, the user doesn't have any plan and they're flexible, the travel arrangement agent will follow the same pattern as the other two described previously. They will all work together, constantly sharing information, and evaluating each other's responses to draft a plan.

What trade-offs did you consider in your design, particularly in terms of reliability, complexity, and coordination overhead? For example, adding feedback loops may improve reliability but increase latency.

- One big one is specialization. More agents means finer and more granular information but for this project, having more than roughly three agents is overkill. Grouping similar tasks, such as travel arrangements, to a single agent is fine, especially given that they share similar constraints and have relatively low complexity.
- Another trade off is reliability versus speed. Allowing agents to read, validate, and revise each other's recommendations can reduce scheduling conflicts and improve the quality of the itinerary. However, repeated feedback loops increase processing time and cost. To combat this, I will limit the number of revisions or stop once all major constraints are satisfied.

How does your proposed architecture support effective and scalable problem solving?

- The current architecture is very modular with three specialized agents. As the app continues to grow, there will be more constraints and features, some of which can be built into another specialized agent and plugged into the existing framework. The agents run in parallel as well, reducing latency as well.